

Resumen del Proyecto

Este documento resume el estado actual del proyecto, su arquitectura, módulos principales, flujos de trabajo, modelos y tablas relevantes, configuración del entorno, pruebas realizadas, problemas resueltos y mejoras pendientes.

1. Resumen Ejecutivo

- Aplicación Laravel (12.32.5) con Livewire para gestión de mantenimientos de maquinarias.
- Flujo de mantenimiento separado en dos fases: creación de la orden y finalización (con modal).
- Soporte para mantenimientos preventivos (con kit preconfigurado) y correctivos (con insumos usados y movimientos de stock automáticos).
- Notificaciones por correo (Mailtrap) ante creación/umbrales y verificación de stock.
- Base de datos PostgreSQL.

2. Stack y Arquitectura

- Backend: Laravel 12.x (PHP 8.2)
- Frontend: Blade + Livewire, Bootstrap
- BD: PostgreSQL
- Email: Mailtrap (SMTP sandbox)
- Jobs/Comandos: Artisan Command para umbrales de mantenimiento

3. Estructura Relevante del Repositorio

- `rennova/app/Http/Livewire/Mantenimientos.php`: Componente Livewire principal para ABM y finalización de mantenimientos.
- `rennova/resources/views/livewire/mantenimientos.blade.php`: Vista del componente con formulario + listado + modal custom de finalización.
- `rennova/resources/views/mantenimientos/index.blade.php`: Carga el componente correcto `@livewire('mantenimientos')`.
- `rennova/app/Console/Commands/CheckMantenimientoUmbrales.php`: Comando que verifica umbrales y genera órdenes + notifica.
- `rennova/app/Models/*`: Modelos de dominio (Mantenimiento, Maquinaria, TipoMantenimiento, Insumo, etc.)
- `rennova/database/migrations/*`: Migraciones, incl. `mantenimiento_insumos` y columnas de costo/subtotal.
- Documentación previa del proyecto en varios `.md` dentro de `rennova/`.

4. Módulos y Funcionalidades

4.1 Mantenimientos (Livewire)

- Crear orden (preventiva/correctiva) con campos: maquinaria, tipo, fecha_inicio, estado (`programado` o `en curso`).
- Visualización de listado con acciones: completar, editar (para no finalizadas), eliminar.
- Finalización mediante modal (overlay propio):

- Campos: `fecha_fin` (requerido), `costo_total` (opcional; se suma automáticamente el costo de insumos si corresponde).
- Si el tipo es correctivo: alta de insumos usados dinámicos (`id_insumo`, `cantidad`, `precio_unitario`).
- Al completar: actualiza la orden, registra insumos y genera movimientos de stock (salida) con motivo.

4.2 Comando de Umbrales

- `CheckMantenimientoUmbrales`: revisa maquinarias en estado `operativa` y sus toneladas acumuladas vs. umbral.
- Crea órdenes preventivas al superar umbral y envía notificación (Mailtrap).
- Verificación de stock: alerta si faltan insumos del kit preventivo.

4.3 Notificaciones por Correo

- Notificación al crear o detectar mantenimientos por umbral (`MantenimientoCreado`) y stock insuficiente.
- Integrado con Mailtrap (tasa limitada en sandbox).

5. Modelos y Tablas Clave

- `Mantenimiento` (tabla `mantenimientos`)
 - PK: `id_mantenimiento`
 - Campos: `id_maquinaria`, `id_tipo_mantenimiento`, `fecha_inicio`, `fecha_fin`, `estado`, `costo_total`, `toneladas_snapshot`, `costo_mano_obra`
 - Relaciones: `maquinaria()`, `tipoMantenimiento()`, `mantenimientoInsumos()`
- `Maquinaria` (tabla `maquinarias`)
 - Campos clave: `estado` (usando `operativa`), `umbral_toneladas`, `toneladas_acumuladas`
- `TipoMantenimiento` (tabla `tipo_mantenimientos`)
 - Ejemplos: `Preventivo`, `Correctivo` (campo `activo`)
- `MantenimientoInsumo` (tabla `mantenimiento_insumos`)
 - PK: `id_mantenimiento_insumo`
 - Campos: `id_mantenimiento`, `id_insumo`, `id_movimiento` (nullable), `cantidad_utilizada`, `costo_unitario`, `subtotal`, `timestamps`
- `MovimientoStock` (tabla `movimiento_stocks`)
 - Campos: `id_movimiento_stock`, `id_insumo`, `tipo` (`salida` para mantenimientos correctivos), `cantidad`, `fecha`, `motivo`
- `Insumo` (tabla `insumos`)
- `KitMantenimientoPreventivo` (tabla `kit_mantenimiento_preventivo`)
 - Campos: `id_maquinaria`, `id_insumo`, `cantidad_requerida`

6. Flujos de Usuario

6.1 Crear Orden

- Ruta: `/mantenimientos` → pestaña "Nuevo Mantenimiento".
- Seleccionar maquinaria + tipo + fecha + estado.
- Si tipo es preventivo, se muestra el kit (si existe) o advertencia para configurarlo.

6.2 Finalizar Orden

- Pestaña "Listado" → botón "Completar" (si no está completada).
- Modal solicita fecha de finalización y opcionalmente costo total.
- Para correctivo: cargar insumos usados (cantidad y precio unitario). El costo total suma automáticamente el subtotal de cada insumo + el costo manual (si se provee).
- Al confirmar: actualiza orden, crea movimientos de stock (salida) y registra mantenimiento_insumos.

7. Reglas de Negocio y Validaciones

- Maquinarias consideradas por el comando: `estado = operativa`.
- Validación de finalización:
 - `fecha_fin ≥ fecha_inicio`.
 - `costo_total` opcional ≥ 0 .
 - Para correctivo, cada insumo usado con $cantidad > 0$ y $precio_unitario \geq 0$ suma al costo total.

8. Configuración y Entorno

- Requisitos: PHP 8.2, Composer, PostgreSQL.
- Email: configurar Mailtrap en `.env` (host, puerto, credenciales sandbox).
- Arranque:
 - `php artisan serve` (o `php -S 127.0.0.1:8000 -t public`).
 - Compilación frontend según corresponda (Vite, si se usa).

9. Pruebas y Utilidades

- Se realizaron pruebas de:
 - Generación de órdenes por umbral.
 - Envío de emails (bloqueo del segundo por límite de Mailtrap en sandbox).
 - Verificación y alertas por stock insuficiente.
 - Flujo de creación y finalización con modal (incluyendo correcciones Livewire y CSS).

10. Problemas Resueltos y Ajustes Técnicos

- Livewire: error "MultipleRootElementsDetected" solucionado envolviendo la vista en un único contenedor raíz.
- Modal: migrado a overlay propio controlado por Livewire para evitar dependencias de Bootstrap JS y conflictos de estado.
- Persistencia del estado del modal: se usa `orden_completar_id` + array `orden_completar_info` y bandera `orden_es_correctivo` (evita pérdida del modelo entre renders).
- Cálculo de costos: `costo_total` se calcula sumando `cantidad × precio_unitario` de insumos correctivos, más un monto manual opcional.
- Columnas de BD: corregidos nombres a `cantidad_utilizada`, `costo_unitario`, `subtotal` según migraciones.
- Comando de umbrales: actualizado para usar `operativa` en vez de `activo`.
- Vista index: carga el componente `mantenimientos` (en vez de `gestion-mantenimientos`).

11. Mejoras Pendientes y Siguietes Pasos

- Validación de stock al completar correctivo (bloquear si no hay stock suficiente o mostrar confirmación).
- Filtros avanzados de listado (por estado, fechas, maquinaria, tipo, rango de costos).
- Reportes: consumo de insumos, costos por maquinaria/periodo, KPIs de mantenimiento.
- Auditoría de cambios (ya hay OwenIt Auditing en Mantenimiento; extender a otras tablas).
- Edición de órdenes no completadas (formulario ya soporta edición básica).
- Envíos de notificación al completar orden (si se requiere flujo de aprobación).
- Tests automatizados (Pest/PHPUnit) para los flujos críticos.

12. Comandos Útiles

- Limpiar cache de vistas: `php artisan view:clear`
- Servir app: `php artisan serve`
- Revisar logs: `tail -f storage/logs/laravel.log` (o en Windows con PowerShell: `Get-Content storage/logs/laravel.log -Tail 100 -Wait`)
- Ejecutar comando de umbrales (si existe una signature definida): `php artisan check:mantenimiento-umbrales` (placeholder; verificar nombre real)

Si necesitás que este documento se separe en guías específicas (p. ej., "Manual de Mantenimientos", "Guía de Configuración de Mailtrap", "Procedimiento de Umbrales"), puedo dividirlo y enlazarlos desde un índice principal.